

// Jivops

Ansible Tower

Guía completa — 20 lecciones

Fundamentos · Recursos · Automatización avanzada · Operación

Contenido

Módulo 1: Fundamentos y acceso

- Lección 01 — Qué es Ansible Tower y Automation Platform
- Lección 02 — Arquitectura y componentes
- Lección 03 — Instalación y despliegue
- Lección 04 — Organizaciones, usuarios y equipos
- Lección 05 — Control de acceso basado en roles

Módulo 2: Recursos y plantillas

- Lección 06 — Inventarios
- Lección 07 — Credenciales
- Lección 08 — Proyectos y control de versiones
- Lección 09 — Job templates
- Lección 10 — Surveys y variables

Módulo 3: Automatización avanzada

- Lección 11 — Workflows
- Lección 12 — Planificación y notificaciones
- Lección 13 — Entornos de ejecución
- Lección 14 — Inventarios dinámicos y sincronización
- Lección 15 — API REST y automatización externa

Módulo 4: Escalado y operación

- Lección 16 — Instance groups y escalado
- Lección 17 — Automation hub y collections
- Lección 18 — Registro, auditoría y análisis
- Lección 19 — Copias de seguridad y actualización
- Lección 20 — Proyecto final: Tower para un equipo de operaciones

Módulo 1: Fundamentos y acceso

Lección 01 — Qué es Ansible Tower y Automation Platform

Una interfaz web, API REST y control de acceso por roles para ejecutar automatización de forma centralizada: inventarios, credenciales y proyectos compartidos, planificación, flujos de trabajo, registro de cada ejecución y auditoría; su versión upstream es AWX.

Centraliza credenciales, aplica permisos por roles y deja registro y auditoría de quién ejecutó qué.

Lección 02 — Arquitectura y componentes

Una interfaz web y API, una base de datos PostgreSQL con el estado y la configuración, nodos de control que planifican trabajos y nodos de ejecución que los corren (comunicados por una malla Receptor), y entornos de ejecución en contenedor con las dependencias.

Permite escalar la capacidad de ejecución y colocar nodos cerca de las redes gestionadas, separando la planificación del trabajo.

Lección 03 — Instalación y despliegue

Con el instalador del paquete de la plataforma (bundle o en línea) y un inventario que define los servidores de cada componente, junto con las contraseñas y la base de datos; también existe despliegue sobre OpenShift con operador.

Cuántos nodos y con qué roles, dónde va la base de datos, la alta disponibilidad y el dimensionamiento.

Lección 04 — Organizaciones, usuarios y equipos

Con organizaciones como contenedor de recursos, usuarios individuales y equipos (teams) que agrupan usuarios; los permisos se asignan a equipos o usuarios sobre los recursos.

Simplifica el mantenimiento: al entrar o salir alguien solo se cambia su pertenencia al equipo.

Lección 05 — Control de acceso basado en roles

Roles sobre cada recurso, como Admin, Execute, Use, Read o Update, de modo que se puede permitir ejecutar una plantilla sin editarla, o usar una credencial sin poder verla.

El principio de mínimo privilegio: cada persona solo recibe los permisos que necesita.

Módulo 2: Recursos y plantillas

Lección 06 — Inventarios

Una colección de hosts y grupos sobre los que se ejecuta la automatización, con variables propias; puede ser estático (definido a mano) o dinámico, obtenido de una fuente como un proveedor de nube.

Se actualiza desde la fuente (por ejemplo la nube) sin mantener la lista a mano.

Lección 07 — Credenciales

Las almacena cifradas, las asocia a plantillas e inventarios sin mostrarlas, y existen tipos para SSH, nube, control de versiones, vault y más; el usuario solo necesita permiso para usarlas.

Para no exponerlas a los usuarios ni dejarlas en los playbooks, y poder rotarlas en un solo sitio.

Lección 08 — Proyectos y control de versiones

Una colección de playbooks y roles, normalmente en un repositorio Git; Tower la sincroniza (con opción de actualizar antes de cada ejecución) para que las plantillas usen siempre la versión elegida de la rama o etiqueta.

Aporta versionado, revisión de cambios e historial, con trazabilidad de lo que se ejecuta.

Lección 09 — Job templates

La combinación de inventario, proyecto, playbook, credenciales y opciones (variables, límite, etiquetas, forks, modo de comprobación) que se lanza una y otra vez, con permisos de ejecución independientes.

Permite acotar la ejecución a un subconjunto de hosts o tareas sin cambiar el playbook.

Lección 10 — Surveys y variables

Para pedir al lanzar una plantilla valores en un formulario (texto, opciones, contraseñas) que se pasan como variables extra, haciendo la automatización reutilizable y controlada sin editar el playbook.

Permite que usuarios no técnicos lancen tareas con parámetros controlados, sin editar el playbook.

Módulo 3: Automatización avanzada

Lección 11 — Workflows

Un encadenamiento visual de plantillas, proyectos, sincronizaciones de inventario y aprobaciones con ramas según el resultado (éxito, fallo o siempre), que permite orquestar procesos de varios pasos con lógica condicional.

Permite reaccionar al resultado: continuar, deshacer cambios o notificar si algo falla.

Lección 12 — Planificación y notificaciones

Con schedules que lanzan plantillas, proyectos o inventarios en fechas y repeticiones definidas, y con notificaciones (correo, Slack, webhooks y otros) al empezar, terminar bien o fallar un trabajo.

Una notificación al canal del equipo cuando el trabajo falle, y opcionalmente al terminar bien.

Lección 13 — Entornos de ejecución

Una imagen de contenedor con Ansible, collections y dependencias Python/sistema que se usa para ejecutar los trabajos, de forma reproducible y aislada, sustituyendo a los antiguos virtualenv de Tower.

Es reproducible y aislado: las dependencias van empaquetadas y se obtiene el mismo resultado en cada nodo.

Lección 14 — Inventarios dinámicos y sincronización

Añadiendo al inventario una fuente de un tipo soportado (AWS, Azure, GCP, VMware, OpenStack, Satellite, un script) con su credencial; la fuente se sincroniza manualmente, al lanzar o por programación, y puede filtrar y agrupar hosts.

Evita ejecutar sobre hosts que ya no existen o no incluir los nuevos por usar datos desactualizados.

Lección 15 — API REST y automatización externa

Mediante su API REST (por ejemplo `/api/v2/`), usando tokens, desde pipelines CI/CD, scripts o la CLI `awx`; también con callbacks de aprovisionamiento y webhooks desde repositorios.

Para lanzar una plantilla de despliegue tras compilar y probar, automatizando el paso a producción.

Módulo 4: Escalado y operación

Lección 16 — Instance groups y escalado

Añadiendo nodos de ejecución y agrupándolos en instance groups, que se asignan a organizaciones, inventarios o plantillas para dirigir los trabajos a nodos concretos (por sede, red o requisito) según capacidad.

Para ejecutar desde un nodo con acceso a una red o sede concreta, cerca de los hosts a gestionar.

Lección 17 — Automation hub y collections

Es el repositorio de contenido de la plataforma: collections certificadas y propias, y entornos de ejecución, de modo que los equipos usan contenido validado en lugar de bajar cualquier cosa de internet.

Control y seguridad: solo se usa contenido validado, incluso en entornos sin salida a internet.

Lección 18 — Registro, auditoría y análisis

Cada trabajo guarda salida, usuario, plantilla, parámetros y estado; el registro de actividad audita cambios, y se pueden enviar logs a un sistema externo y usar herramientas de analítica para medir uso y ahorro.

Permite saber quién ejecutó qué y cuándo, para investigar incidencias y exigir responsabilidades.

Lección 19 — Copias de seguridad y actualización

Con las copias del instalador (o de la base de datos y los secretos como SECRET_KEY), probando la restauración, y actualizando mediante el instalador siguiendo la ruta soportada y tras revisar las notas de la versión.

Tener copia probada, leer las notas de la versión y revisar la compatibilidad de entornos y collections.

Lección 20 — Proyecto final: Tower para un equipo de operaciones

Nodos de control con alta disponibilidad y PostgreSQL protegida, nodos de ejecución en cada sede en instance groups, una organización por equipo con teams y RBAC, proyectos en Git, credenciales centralizadas, inventarios dinámicos, entornos de ejecución propios desde un hub, workflows con aprobación, notificaciones y copias probadas.

Cambio en Git con revisión, prueba en un entorno previo, y un workflow con aprobación antes de producción.